

WORLD ROBOTICS OLYMPIAD 2026
TEAM HERMES
TECHNICAL DOCUMENTATION

Table of Contents

1. Mobility & Mechanical Design.....	1
2. Power & Sensor Architecture	6
3. Software Architecture & Obstacle Strategy	15
4. Systems Thinking & Engineering Decisions	26
5. Reproducibility & GitHub Quality.....	33
Appendix A - Testing and Design Iterations.....	40
Appendix B - Mechanical Drawings, Wiring and CAD References	42
Appendix C - Bill of Materials	43

1. Mobility & Mechanical Design

1.1 Design Objectives

The primary objective of Hermes is to provide a reliable, autonomous vehicle capable of completing the WRO 2026 Future Engineers challenge while remaining fully compliant with all competition regulations. Throughout the mechanical design process, emphasis was placed on reliability, modularity, ease of maintenance, and repeatable performance rather than maximum speed.

The robot was designed to:

- Maintain stable steering during continuous autonomous driving.
- Navigate both straight sections and corners with minimal oscillation.
- Provide sufficient space for all electronic components while remaining within the competition size constraints.
- Allow rapid replacement or adjustment of hardware during testing.
- Support future modifications without requiring a complete chassis redesign.

The design philosophy adopted by the team prioritizes consistency over aggressive performance, since a slower but reliable robot is expected to achieve a higher competition score than a faster but inconsistent one.

1.2 Mechanical Configuration

Hermes uses a conventional four-wheel vehicle architecture consisting of:

- Four wheels
- One powered drive axle
- One steering axle
- A rigid chassis supporting all electronic and mechanical components

This configuration satisfies the mechanical requirements specified in the WRO 2026 Future Engineers rules.

The steering system is physically separated from the propulsion system. Steering is achieved using a servo motor that rotates the front wheels through a steering linkage, while propulsion is provided by a

DC motor driving a single axle. Separating steering and propulsion simplifies both the mechanical design and the software control architecture.

The overall chassis was designed to provide:

- Low centre of gravity
- Even weight distribution
- Adequate rigidity
- Easy access to electronics during testing
- Secure mounting locations for sensors and structural components



1.3 Chassis Design

The chassis serves as the structural foundation for every subsystem of the robot.

It provides mounting locations for:

- Raspberry Pi 4
- ESP-WROOM-32 microcontroller
- Camera module
- IMU
- Time-of-Flight sensors
- Battery pack
- Steering servo
- Drive motor
- Wiring and power distribution components

The chassis layout was selected to reduce vibration reaching the vision system while also protecting electronics from accidental impact during testing.

Structural rigidity was considered important because excessive chassis flex introduces steering inaccuracies and inconsistent sensor alignment. A rigid frame improves repeatability between runs by maintaining constant sensor positions relative to the vehicle.

Several components are mounted using dedicated brackets or custom 3D-printed parts to simplify maintenance and replacement.

1.4 Steering System

Hermes employs an Ackermann-style steering arrangement driven by a single MG90S metal gear servo motor.

This design was selected because it provides:

- Accurate steering angle control

- Fast response time
- Repeatable positioning
- Low mechanical complexity
- Compliance with the single steering actuator rule

Unlike differential steering systems, servo steering allows the robot to maintain smooth trajectories while preserving traction during cornering.

Steering commands are generated by the Raspberry Pi and transmitted to the ESP32, which converts steering requests into servo angles through PWM control.

Using a dedicated steering actuator also simplifies PID tuning since steering angle can be directly controlled rather than indirectly through wheel speed differences.

1.5 Drive System

Vehicle propulsion is provided by a single DC motor connected to one drive axle.

The drive system was chosen because it offers:

- Simple control
- High reliability
- Reduced mechanical complexity
- Compliance with WRO regulations requiring a single drive axle

Motor speed is controlled electronically by the ESP32 using PWM signals generated from commands received from the Raspberry Pi.

Separating high-level decision making from low-level motor control reduces latency and allows precise speed regulation without overloading the main processor.



1.6 Wheelbase and Stability

Mechanical stability was considered throughout the chassis design.

A longer wheelbase generally provides:

- Improved straight-line stability
- Reduced steering oscillations

- More predictable cornering behaviour

However, increasing wheelbase also increases turning radius.

The final design represents a compromise between:

- Stability
- Turning capability
- Available chassis space
- Competition size constraints

Similarly, the vehicle width was selected to provide adequate lateral stability while remaining comfortably within the maximum allowed dimensions.

1.7 Component Mounting Strategy

Sensor placement was carefully considered to maximize measurement accuracy while minimizing interference between subsystems.

The camera is mounted at the front of the vehicle with an unobstructed field of view toward the track.

The IMU is positioned near the centre of mass to reduce rotational measurement errors caused by vibration.

Time-of-Flight sensors are mounted to provide consistent distance measurements relative to surrounding track boundaries.

The battery pack is positioned as low as practical to reduce the centre of gravity and improve stability during cornering.

The Raspberry Pi and ESP32 are mounted securely to minimize vibration while remaining easily accessible for programming and debugging.

1.8 Mechanical Design Trade-offs

Several design alternatives were considered before selecting the final configuration.

	Steering Mechanism	Drive Configuration
Alternative	Differential steering using independently driven wheels.	Four-wheel drive.
Rejection Reason	▪ Does not comply with WRO Future Engineers requirements. ▪ Produces larger turning radii. ▪ Complicates trajectory control.	▪ Increased mechanical complexity. ▪ Higher power consumption. ▪ Additional drivetrain losses. ▪ Not required for the competition environment.
Selected Solution	Single-servo steering with mechanical linkage.	Single powered axle.
Reason	▪ Rule compliant. ▪ More accurate steering. ▪ Easier to tune using PID control. ▪ More predictable vehicle behaviour.	▪ Simpler implementation. ▪ Lower current draw. ▪ Easier maintenance. ▪ Satisfies competition requirements.

Modular Construction

Rather than permanently fixing every component to the chassis, the team adopted a modular construction philosophy.

This allows:

- Faster hardware replacement
- Easier debugging
- Sensor repositioning during calibration
- Simplified future upgrades

This approach proved particularly valuable during iterative development, where hardware frequently required adjustment after software modifications.

1.9 Mechanical Testing and Iteration

Mechanical development followed an iterative design process.

Each iteration focused on improving one or more of the following:

- Steering accuracy
- Vehicle stability
- Component accessibility
- Structural rigidity
- Sensor alignment
- Reliability during repeated autonomous runs

Changes were validated through repeated testing before being incorporated into the final design.

1.10 Compliance with WRO Requirements

The mechanical design was developed specifically to satisfy the WRO 2026 Future Engineers regulations.

Compliance checklist:

- ✓ Four-wheel vehicle
- ✓ Single drive axle
- ✓ Single steering actuator
- ✓ Fully autonomous operation
- ✓ No wireless communication
- ✓ Modular hardware design

1.11 Summary

The Hermes mechanical design emphasizes simplicity, reliability, and maintainability. Rather than maximizing speed, the team prioritized repeatable autonomous behaviour through a rigid chassis, servo-based steering, modular construction, and a conventional drive architecture. These decisions simplify both hardware integration and software control while ensuring compliance with the WRO 2026 Future Engineers regulations. Future mechanical refinements will primarily focus on improving durability, steering precision, and ease of maintenance without significantly altering the overall system architecture.

2. Power & Sensor Architecture

2.1 Design Objectives

The power and sensor architecture of Hermes was designed to provide reliable operation throughout an autonomous run while maintaining a clear separation between high-level perception, low-level control, and power distribution.

The architecture was developed with four primary objectives:

- Provide stable power to all electronic components.
- Ensure sensors receive consistent and reliable measurements.
- Minimize electrical noise affecting perception and control.
- Simplify troubleshooting through modular wiring.

Rather than optimizing for the lowest possible weight or highest possible performance, the team prioritized robustness and ease of maintenance.

2.2 System Architecture

Hermes uses a distributed computing architecture consisting of two processing units.

Raspberry Pi 4

The Raspberry Pi serves as the high-level computer responsible for:

- Computer vision
- State machine execution
- Lane following
- Obstacle detection
- Motion planning
- Communication with the ESP32

ESP-WROOM-32

The ESP32 performs all real-time control tasks including:

- Servo control
- Motor speed control
- PWM generation
- Safety monitoring
- Command execution

Separating these responsibilities prevents computationally intensive vision algorithms from affecting real-time motor control.

2.3 Power Architecture

The robot is powered using rechargeable lithium-ion batteries arranged as a battery pack.

The power system supplies energy to:

- Raspberry Pi 4
- ESP-WROOM-32
- Steering servo
- Drive motor
- Camera

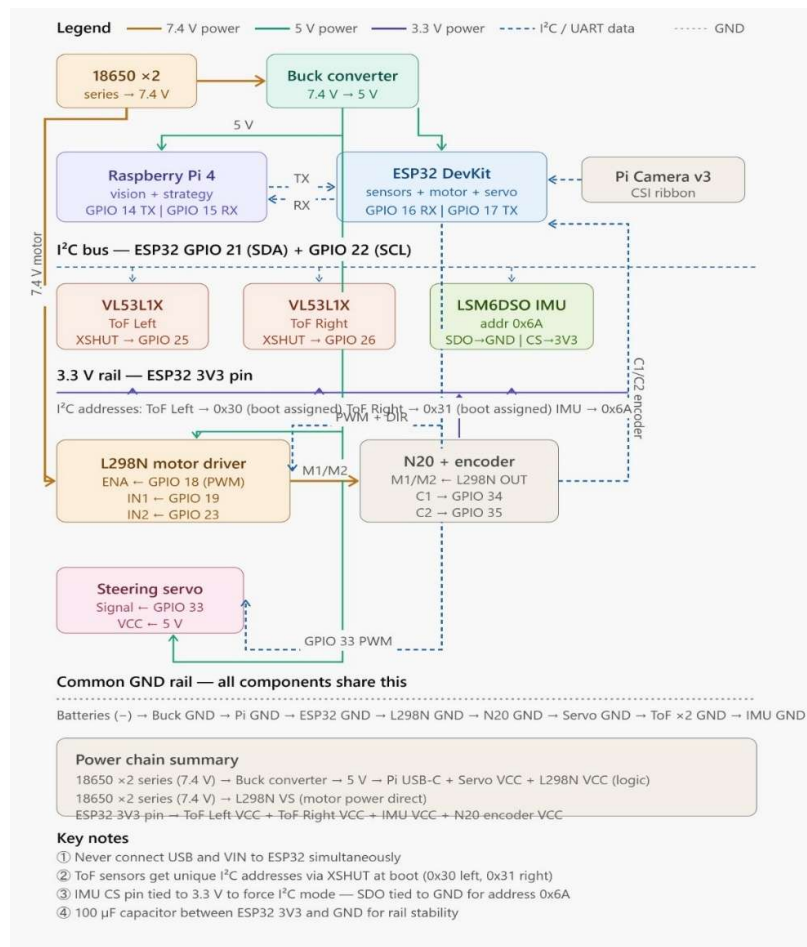
- IMU
- Time-of-Flight sensors

Because these devices operate at different voltage levels and have different current requirements, regulated power conversion is used to ensure stable operation.

A buck converter is used to provide a regulated supply voltage for components requiring lower operating voltages.

This architecture minimizes voltage fluctuations while protecting sensitive electronics from battery voltage variations.

Power Distribution Diagram



2.4 Battery Selection

Hermes uses lithium-ion 18650 rechargeable cells due to their:

- High energy density
- Rechargeability
- Wide availability
- Stable discharge characteristics
- Good capacity-to-weight ratio

The battery pack was selected to provide sufficient operating time for:

- Practice sessions
- Calibration
- Multiple competition runs

without requiring frequent battery replacement.

Measured Battery Capacity

A 3S Li-ion pack has nominal 11.1 V (3×3.7 V) and fully charged 12.6 V (3×4.2 V). For cell capacities of 2200, 3000, and 3500 mAh, the battery energy is:

- **Nominal (11.1 V):**
 - 2200 mAh: $11.1 \text{ V} \times 2.2 \text{ Ah} = 24.420 \text{ Wh}$
 - 3000 mAh: $11.1 \text{ V} \times 3.0 \text{ Ah} = 33.300 \text{ Wh}$
 - 3500 mAh: $11.1 \text{ V} \times 3.5 \text{ Ah} = 38.850 \text{ Wh}$
- **Full charge (12.6 V):**
 - 2200 mAh: $12.6 \text{ V} \times 2.2 \text{ Ah} = 27.720 \text{ Wh}$
 - 3000 mAh: $12.6 \text{ V} \times 3.0 \text{ Ah} = 37.800 \text{ Wh}$
 - 3500 mAh: $12.6 \text{ V} \times 3.5 \text{ Ah} = 44.100 \text{ Wh}$

Expected Operating Time

$$\text{Runtime (hrs)} = \frac{\text{Battery Energy (Wh)}}{\text{Total Power (W)}}$$

Using nominal 11.1 V:

- **2200 mAh (24.420 Wh):**
 - Runtime $\approx 24.420 / 20.600 = 1.186 \text{ hr} (\approx 1 \text{ h } 11 \text{ min})$
- **3000 mAh (33.300 Wh):**
 - Runtime $\approx 33.300 / 20.600 = 1.616 \text{ hr} (\approx 1 \text{ h } 37 \text{ min})$
- **3500 mAh (38.850 Wh):**
 - Runtime $\approx 38.850 / 20.600 = 1.888 \text{ hr} (\approx 1 \text{ h } 53 \text{ min})$

2.5 Voltage Regulation

Different electronic components require different supply voltages.

Rather than powering every component directly from the battery pack, regulated voltage conversion is used to provide stable operating conditions.

Voltage regulation offers several advantages:

- Improved system stability
- Reduced electrical noise
- Protection against battery voltage variation
- More reliable sensor measurements
- Reduced likelihood of processor resets

Stable voltage regulation is particularly important for the Raspberry Pi and ESP32 because voltage drops can lead to unexpected software failures or system restarts during autonomous operation.

2.6 Estimated Power Budget

The overall power budget was considered during system design to ensure that the selected battery pack could safely supply every subsystem.

A detailed power budget (Table below) lists each subsystem's voltage, current, and power. Values are from official sources when available. Notable points: the Raspberry Pi 4 draws ~0.540 A (2.7 W) at idle and up to ~1.280 A (6.4 W) under load [2+L38-L41]. We assume ~0.900 A (4.500 W) as a design point. The ESP32 DevKit typically consumes ~80 mA at 3.3 V [14+L184-L188] (0.264 W); accounting for 5 V logic and USB interface, we assume ~120 mA at 5 V (0.600 W). The Raspberry Pi Camera Module (~1.25 W) needs about 0.250 A from 5 V [16+L179-L187]. Each VL53L1X TOF sensor draws ~19 mA at 3.3 V (0.063 W) [22+L301-L308]. The LSM6DSO IMU in high-performance mode draws ~0.55 mA at 3.3 V [24+L2-L10] (~0.002 W). A typical 5 V hobby servo (e.g. SG90/MG90) may draw ~0.550–0.750 A at stall [26+L43-L47]; we assume 0.500 A (2.500 W) for sustained load. The L298N driver's logic supply (~5 V) draws on the order of 30–50 mA [30+L200-L209]. The N20 geared DC motors (running at ~11.1 V) draw roughly 0.8 A each under load (~8.880 W) [32+L21-L24] [33+L153-L161]; their no-load current is only ~0.04–0.08 A [32+L21-L24] [33+L153-L161]. Encoder and minor GPIO loads are negligible.

Component	Voltage (V)	Current (A)	Power (W)
Raspberry Pi 4	5.000	0.900	4.500
ESP32 DevKit	5.000	0.120	0.600
Pi Camera v3	5.000	0.250	1.250
VL53L1X Left	3.300	0.020	0.066
VL53L1X Right	3.300	0.020	0.066
LSM6DSO IMU	3.300	0.0006	0.002
Steering Servo	5.000	0.500	2.500

2.7 Wiring Strategy

The wiring architecture was designed to reduce clutter while simplifying maintenance and fault diagnosis.

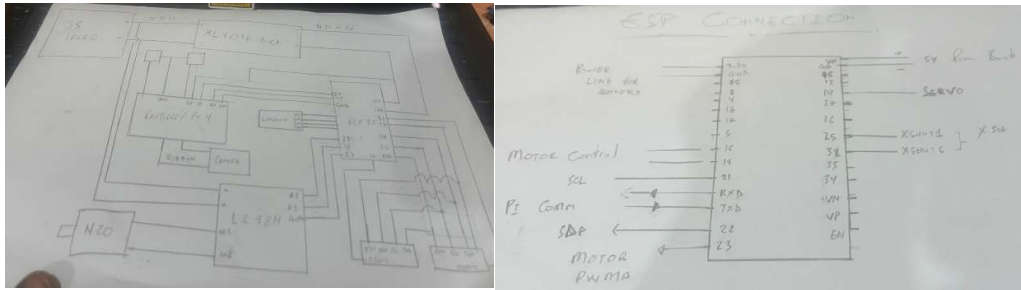
Key principles used include:

- Short signal paths where practical.
- Separation of power wiring from communication wiring.
- Secure mechanical fastening of connectors.
- Modular wiring to simplify component replacement.

Communication between the Raspberry Pi and ESP32 is performed using a dedicated serial connection, allowing high-level motion commands to be transmitted without introducing unnecessary wiring complexity.

Power distribution follows a centralized architecture, with regulated outputs supplying each subsystem independently.

Wiring Diagram (System and ESP)



2.8 Sensor Selection

Hermes combines multiple complementary sensors to improve robustness during autonomous navigation.

Raspberry Pi Camera Module 3

The camera serves as the primary perception sensor.

It is responsible for:

- Detecting coloured pillars
- Identifying lane boundaries
- Supporting track following
- Detecting finish and parking regions

The camera provides significantly richer environmental information than individual distance sensors and therefore forms the foundation of the perception pipeline.

Inertial Measurement Unit (IMU)

The IMU (LSM6DSOX, 6-axis accel+gyro) is physical hardware wired only to the ESP32's I2C bus, so, the Pi never opens an I2C bus or reads a register itself; the ESP32 streams ax,ay,az,gx,gy,gz up in every TEL packet, and hardware/imu.py only ever reads that already-published value back out of the TelemetryHub. The one computation the Pi does perform is integrating gz (yaw rate) over dt into a running heading estimate - this is a control-layer computation over already-transmitted data, not a hardware read, which is why it lives on the Pi alongside the rest of the decision-making in control/steering_control.py. That heading estimate is the single input to the heading-hold PID: FOLLOW_TRACK locks a target heading once (hold_straight()) and holds it through pure IMU integration by default, AVOID_OBSTACLE locks an offset target via turn_by() when a pillar is detected, and the camera's lane-centering and TOF-based wall-avoidance both work by nudging that same locked target (nudge_target()) rather than issuing their own steering commands - so the IMU heading-hold loop is the single point that actually talks to the servo, and everything else (vision, ToF) only ever adjusts what it's aiming for.

Time-of-Flight Sensors

Two Time-of-Flight sensors are mounted on the robot.

Their intended purpose is to:

- Measure distance to nearby walls
- Improve positioning accuracy
- Provide additional environmental awareness
- Support safe navigation around obstacles

Using active distance sensors complements vision by providing direct distance measurements that are less sensitive to lighting conditions.

Both ToF sensors (VL53L0X) are physical hardware wired only to the ESP32, sharing the IMU's I2C bus and re-addressed at boot via the standard XSHUT sequence (VL53L0X has no hardware address-select pin, so the LEFT sensor is initialised alone and moved off the shared default 0x29 before the RIGHT sensor is released from reset). `firmware/esp_controller/tof.cpp`'s `readTOF()` fills `tof1_mm(left)/tof2_mm(right)` into the same `RobotTelemetry` struct the IMU uses, sent up as one TEL packet - there is no separate ToF packet type. On the Pi, `hardware/tof.py` is a pure pass-through reading the latest published telemetry, with no sensor logic of its own. Integration into decision-making happens entirely in `planning/obstacle_planner.py`: `adjust_avoidance_steer()` caps the pillar-avoidance steering angle (and issues a small corrective heading nudge) when the ToF on the side being steered toward reads below a warning/critical distance, and `general_speed_scale()` derives a 0.5–1.0 speed multiplier from whichever side wall is closest, applied continuously in both `FOLLOW_TRACK` and `AVOID_OBSTACLE`. This makes ToF a safety guardrail layered on top of camera-based pillar avoidance, not an independent avoidance system - it cannot change which side a pillar is dodged on, only how far the bot commits to that dodge.

2.9 Sensor Placement

Sensor placement was determined according to the geometry of the WRO playing field.

Camera Position

The vision system uses a Raspberry Pi Camera v3 mounted approximately 120 cm above the ground and tilted downward by 20°. This configuration was selected after evaluating the trade-off between field of view, obstacle visibility, and image distortion.

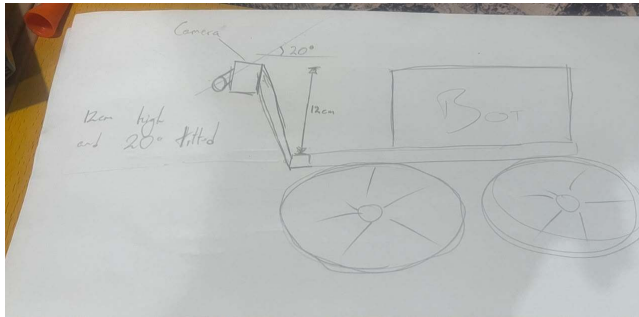
The elevated mounting position increases the visible area in front of the vehicle, allowing the camera to simultaneously observe lane boundaries, wall segments, corner markers, and obstacles at greater distances. A higher viewpoint reduces the likelihood of nearby objects obscuring important navigation features and provides more time for the control algorithm to react to upcoming track elements.

A downward tilt of 20° was chosen to focus the camera on the region most relevant to navigation. With a smaller tilt angle, a significant portion of the image would contain the horizon and background, reducing the number of useful pixels dedicated to the track. Conversely, a much steeper angle would reduce the forward look-ahead distance and make obstacle detection less reliable. The 20° angle provides a balance between near-field visibility and long-range perception.

This configuration offers several advantages:

- Increased look-ahead distance, improving path planning and steering decisions.
- Better obstacle detection, as obstacles remain visible for a longer period before reaching the robot.
- Improved lane recognition, with both track boundaries visible within a single frame.
- Reduced image distortion compared to extreme camera angles.
- More stable computer vision performance, since lighting changes and distant background objects occupy a smaller portion of the image.

As a result, the selected camera position maximizes the amount of useful environmental information available to the vision algorithm while maintaining reliable detection of walls, obstacles, and navigation markers throughout the competition course.



IMU Position

The IMU is mounted close to the vehicle's centre of mass.

This minimizes rotational acceleration errors and reduces measurement noise caused by vibration.

TOF Sensor Placement

The Time-of-Flight sensors are positioned to observe nearby track boundaries while minimizing interference from other components.

Their placement was selected to maximize useful distance measurements during normal driving. They were placed on the two sides of the bot.

2.10 Sensor Calibration

Reliable autonomous behaviour requires careful sensor calibration before testing.

Camera Calibration

The vision pipeline requires calibration of:

- Colour thresholds
- Exposure
- White balance
- Region-of-interest boundaries

Calibration ensures consistent colour detection under varying lighting conditions.

TOF Calibration

Distance sensors require calibration to verify:

- **Detection range:** with each sensor un-obstructed, confirm readings stay near TOF_MAX_VALID_MM (2000mm, config/thresholds.py) rather than drifting or reading spuriously short — VL53L0X's usable range is roughly 30–2000mm depending on target reflectivity.
- **Distance accuracy:** place a flat white target (matching the mat's wall material) at 5-6 known distances spanning the operating range (e.g. 50, 100, 200, 500, 1000mm), log 20+ consecutive readings at each, and compare the mean against the ruler-measured true distance. VL53L0X is typically accurate to a few mm under 200mm, degrading at longer range.
- **Measurement consistency:** at a single fixed distance, compute the standard deviation across 50+ consecutive samples - this quantifies sensor noise the control loop will need to tolerate (informs whether TOF_WALL_WARNING_MM/TOF_WALL_CRITICAL_MM need a larger safety margin).

- **Sensor alignment:** confirm each sensor's optical axis is perpendicular to the side it's meant to measure - an angled mount reads a false, larger distance to a flat wall, which would make the bot think it has more clearance than it does. Checked by comparing the reading against a physically-measured perpendicular distance at a known wall angle.
- **Cross-talk check:** with both sensors active, confirm the left sensor's reading doesn't change when only the right sensor's line of sight is obstructed (and vice versa) - relevant because both sensors share one I2C bus and sit close together on a small chassis.

IMU Calibration

IMU calibration is performed automatically on every boot rather than once at the bench: during INIT, before the start button is armed, the ESP32 holds the chassis in a stationary window and samples raw accelerometer and gyroscope output at the telemetry rate, computing the mean of each axis as that boot's bias (with the accelerometer's vertical axis offset from the expected 1g rather than zero). This bias is subtracted from every subsequent sample before it is packed into the TEL packet, so the Pi's heading-hold controller always operates on already-corrected data and requires no calibration logic of its own. Automatic re-calibration on every power-on was chosen over a fixed bench-measured constant because MEMS gyro bias drifts with temperature and mounting stress, and a constant measured once in a lab would not hold at competition conditions. Sign and scale correctness - confirming that a physical clockwise rotation produces a positive, correctly-scaled heading change, matching the "+ = right" convention used throughout the steering stack - is verified separately via a one-time bench turntable test per hardware build, since that property depends on physical wiring and mounting orientation rather than on drift, and does not need to be re-checked every run.

2.11 Reliability Considerations

Several potential failure modes were considered during system design.

Voltage Drop

Rapid motor acceleration may temporarily increase current consumption.

Mitigation:

- Regulated power supply
- Adequate battery capacity
- Separate control electronics from high-current devices where practical

Sensor Failure

If a sensor produces invalid data, the software architecture is designed to avoid unsafe behaviour.

Fault	Detection	Handling
ESP32 sensor init failure (IMU or either ToF)	Firmware initIMU()/initTOF() return false; ESP32 sends STATUS,ERR ...	Non-fatal on the firmware side - readIMU()/readTOF() fall back to safe defaults (level/stationary, or OUT_OF_RANGE_MM) rather than garbage values; the Pi never even hears an "OK" for that sensor
ESP32 alive but sensors never confirmed	Serial link up, but no valid TEL packet seen	Runtime. await_first_telemetry() blocks briefly at startup; run_startup_diagnostics() reports sensor status as UNCONFIRMED rather than

		asserting REAL, and StateMachine.__init__ gates INIT → WAIT_FOR_START on camera_ready/serial_ready
Malformed/truncated TEL packet	communication/packet_parser.py checks field count and that every field parses as float	Logs a warning, returns {"type": "unknown"}; the last-known-good telemetry in the hub is left untouched rather than being corrupted
Serial port physically unavailable	SerialLink.connect() exception from pyserial	Degrades to EMULATION mode (TX logged, RX empty) instead of crashing the whole process
No camera hardware	Camera._open_real() exceptions from both Picamera2 and OpenCV	Falls back to SYNTHETIC frames; run_startup_diagnostics() reports this honestly rather than pretending the feed is real
A pillar/corner/parking-zone/wall reading vanishes mid-detection	PillarDetector/CornerTracker require N consecutive lost frames (hysteresis) before declaring "cleared", instead of one dropped frame flipping state	Prevents a single noisy frame from double-counting a corner or prematurely exiting AVOID_OBSTACLE
Wall gets dangerously close during a pillar-avoidance turn	planning/obstacle_planner.py checks live ToF against TOF_WALL_CRITICAL_MM	Caps the commanded avoidance angle and issues a corrective nudge away from that wall, every tick, independent of the camera
Invalid state transition requested	TransitionManager.get_next_state() has no matching table entry	Logged, current state left unchanged rather than the state machine silently jumping somewhere undefined
Any FAILURE_DETECTED event, from any state	transitions.py special-cases this event type ahead of the normal lookup table	Unconditionally forces State.ERROR regardless of current state, and Runtime._drain_events() calls self.steering.reset() on entry so no stale PID/target-heading state survives into whatever run follows a manual reset
Uncaught exception anywhere in the tick loop	run_forever()'s top-level try/except Exception	Logged with full traceback, then finally: self.shutdown() fires an EMG (emergency stop) packet before the process exits

Loose Connections

Repeated testing may loosen connectors.

Mitigation strategies include:

- Secure cable routing
- Mechanical strain relief

- Modular connectors
- Routine inspection before testing sessions

2.12 Design Trade-offs

Several design decisions involved balancing competing engineering priorities.

Camera vs Distance Sensors

Rather than relying exclusively on one sensing technology, Hermes combines vision and distance sensing.

Advantages include:

- Improved robustness
- Complementary measurements
- Reduced sensitivity to individual sensor limitations

Distributed Control

Instead of using a single processor for all tasks, Hermes separates perception and real-time control.

Advantages include:

- Better computational efficiency
- Reduced latency
- Easier software maintenance
- Improved system reliability

Battery Capacity vs Weight

Increasing battery capacity extends operating time but also increases vehicle mass.

The selected battery configuration represents a balance between:

- Competition runtime
- Vehicle weight
- Available installation space

2.13 Summary

The power and sensor architecture of Hermes was designed to provide reliable autonomous operation through stable power distribution, modular wiring, and complementary sensing technologies. The Raspberry Pi performs computationally intensive perception and planning tasks, while the ESP-WROOM-32 provides deterministic real-time control of the vehicle. Careful sensor placement, regulated power delivery, and modular system design simplify maintenance while improving overall robustness. Future work will focus on completing sensor calibration, validating the power budget through experimental measurements, and further improving fault tolerance through software-based diagnostics.

3. Software Architecture & Obstacle Strategy

3.1 Design Objectives

The Hermes software architecture was designed around the principle of separating high-level autonomous decision-making from low-level hardware control. This separation improves reliability,

simplifies debugging, and allows each processor to specialize in tasks that match its computational strengths.

The software architecture was developed with the following objectives:

- Maintain modular and reusable code.
- Ensure deterministic robot behaviour.
- Separate perception from control.
- Allow independent testing of software modules.
- Enable safe recovery from communication or sensor failures.

Rather than relying on complex artificial intelligence techniques, Hermes uses deterministic algorithms that are easier to understand, debug, and reproduce.

3.2 Overall Software Architecture

The software is divided between two processing units.

Raspberry Pi

The Raspberry Pi is responsible for all computationally intensive tasks, including:

- Image acquisition
- Computer vision
- Lane detection
- Obstacle detection
- State machine execution
- Motion planning
- Communication with the ESP32

ESP-WROOM-32

The ESP32 performs all real-time hardware control, including:

- Servo control
- Motor speed control
- PWM generation
- Command execution
- Safety watchdog monitoring

This separation ensures that delays in image processing cannot interfere with precise motor control.

3.3 Layered Software Design

The Hermes software follows a layered architecture consisting of four logical layers.

Sensor Layer

Collects raw information from:

- Camera
- IMU
- Time-of-Flight sensors

↓

Perception Layer

Processes raw sensor data into meaningful information such as:

- Track position
- Pillar colour
- Lane offset
- Finish zone detection

↓

Decision Layer

Determines the appropriate robot behaviour using a finite state machine.

↓

Control Layer

Converts high-level commands into steering angles and motor speeds before transmitting them to the ESP32.

This separation improves maintainability and allows each layer to be tested independently.

3.4 Repository Structure

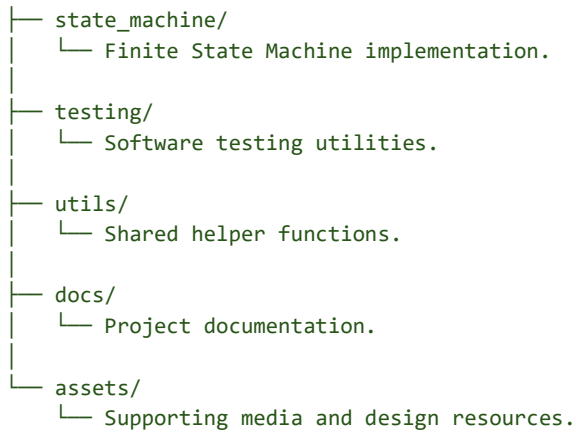
The repository is organized into logical modules that separate responsibilities.

Firmware/

ESP32 firmware responsible for:

- Motor control
- Servo control
- Serial communication
- Safety watchdog
- Hardware interfaces

```
src/
├── communication/
│   └── Configuration and packet handling.
├── config/
│   └── Robot configuration and system parameters.
├── control/
│   └── Steering control, speed control, and drive command generation.
├── hardware/
│   └── Hardware interface modules.
├── perception/
│   └── Computer vision and environmental sensing.
├── planning/
│   └── Navigation and obstacle planning.
├── simulation/
│   └── Simulation utilities.
```



The modular repository structure improves readability and makes future development easier.

3.5 Software Modules

Communication

Responsible for communication between the Raspberry Pi and ESP32.

Responsibilities include:

- Packet formatting
- Command transmission
- Packet parsing
- Communication validation

Commands generated by the Raspberry Pi are transmitted over serial and interpreted by the ESP32 before being converted into motor actions.

Perception

The perception module transforms camera images into information that can be understood by the robot.

Responsibilities include:

- Image acquisition
- Colour segmentation
- Region-of-interest processing
- Contour detection
- Lane estimation
- Pillar detection
- Finish detection

Rather than processing the entire image equally, Regions of Interest (ROIs) are used to reduce computational cost and improve detection speed.

Planning

The planning layer converts perception results into navigation decisions.

Responsibilities include:

- Lane centreing

- Obstacle planning
- Parking planning
- Motion target generation

Planning does not directly control motors.

Instead, it determines the desired steering and speed that should be executed by the control layer.

Control

The control module converts planned motion into actuator commands.

Responsibilities include:

- PID steering
- Speed regulation
- Steering smoothing
- Command generation

Outputs are transmitted to the ESP32 through the communication layer.

State Machine

The state machine controls the overall behaviour of the robot.

Instead of allowing every module to make independent decisions, all behaviour is coordinated through a finite set of predefined operating states.

This ensures predictable and reproducible robot behaviour.

3.6 State Machine Design

Hermes uses a finite state machine to coordinate every stage of autonomous operation.

At any point during execution, the robot exists in exactly one state.

STATE	GOAL	SENSORS	OUTPUT	ENTRY CONDITION	EXIT CONDITION	NEXT STATE (TRANSITION)
INIT	Initialize all systems safely	Camera, IMU, TOF, serial status	System checks, idle motor commands	Robot powered on	All systems initialized successfully	WAIT_FOR_START
WAIT_FOR_START	Wait legally for competition start	Start button	Motors disabled	INIT completed	Start button pressed after judge signal("Go")	FOLLOW_TRACK
FOLLOW_TRACK	Navigate the track while staying centered and moving in correct direction	Camera, TOF, IMU, Wall detection	Steering, speed commands	Start pressed //OR obstacle completed	Pillar detected //OR lap marker detected //OR finish sequence triggered //OR critical failure	AVOID_OBSTACLE //OR ERROR //OR PARK

AVOID_OBSTACLE	Pass obstacle on correct side based on pillar color	Camera, TOF	Modified steering trajectory	Red or green pillar detected	Obstacle successfully passed	FOLLOW_TRACK
LAP_CHECK	Verify and count completed laps	Camera / section detection	Increment lap counter	Lap marker or start section detected	Lap count updated	FOLLOW_TRACK //OR FINAL_APPROACH
FINAL_APPROACH	Return safely to finish/start section after lap completion	Camera	Reduced speed steering control	lap_count == 3	Finish section detected	PARK //OR STOP
PARK	Stop fully within parking/finish zone	Camera	Fine steering, lower speed	Finish section reached	Vehicle fully stopped within target area	STOP
STOP	Maintain complete stop	watchdog monitoring (maybe)	Zero motor speed	Parking complete //OR finish achieved	N/A	N/A
ERROR	Enter safe fail-state	All diagnostic inputs	Stop Motors	Critical failure detected	Manuel reset only	INIT

3.7 Event-Driven Behaviour

State transitions occur only when specific events are detected.

Examples include:

- START_BUTTON_PRESSED
- PILLAR_RED_DETECTED
- PILLAR_GREEN_DETECTED
- OBSTACLE_CLEARED
- LAP_MARKER_DETECTED
- THREE_LAPS_COMPLETE
- FINISH_ZONE_DETECTED
- SERIAL_TIMEOUT
- SENSOR_FAILURE

Using events rather than continuously changing behaviour prevents unexpected transitions and improves software reliability.

3.8 Track Following Strategy

Hermes primarily follows the track using computer vision.

The perception system identifies the track boundaries within predefined Regions of Interest.

These boundaries are used to estimate the robot's lateral position relative to the centre of the lane.

An error value is computed representing the offset between the desired position and the measured position.

This error becomes the input to the steering controller.

Using continuous lane estimation provides smoother motion than discrete steering commands.

```
FRAME_WIDTH := camera.width (px)
center      := FRAME_WIDTH // 2

# In a fixed ROI band (LANE_ROI_TOP_FRAC..LANE_ROI_BOTTOM_FRAC of frame height):
# mask := HSV pixels with V <= LANE_BLACK_V_MAX and S <= LANE_BLACK_S_MAX
# col_counts[x] := smoothed count of masked pixels in column x
# min_pixels := roi_height * LANE_MIN_WALL_FRACTION

left_wall_px := nearest x < center where col_counts[x] >= min_pixels, scanning outward from
center (or None)
right_wall_px := nearest x > center where col_counts[x] >= min_pixels, scanning outward from
center (or None)

if left_wall_px and right_wall_px:
    corridor_center := (left_wall_px + right_wall_px) / 2      # confidence = 1.0
elif left_wall_px only:
    corridor_center := left_wall_px + LANE_DEFAULT_HALF_WIDTH_PX  # confidence = 0.5
elif right_wall_px only:
    corridor_center := right_wall_px - LANE_DEFAULT_HALF_WIDTH_PX  # confidence = 0.5
else:
    lane_offset_px := invalid (no correction applied this frame)

lane_offset_px := clamp(corridor_center - center, ±(FRAME_WIDTH/2 * LANE_MAX_OFFSET_FRAC))
```

3.9 Steering Control

Steering is controlled using a PID controller.

The PID controller continuously minimizes the lane offset error by adjusting the steering angle.

The proportional term provides immediate correction.

The integral term compensates for persistent errors.

The derivative term reduces oscillation by responding to changes in error.

This combination produces smooth steering while reducing overshoot during cornering.

Configured PID gains

Parameter	Value
STEER_PID_KP	1.6
STEER_PID_KI	0.02
STEER_PID_KD	0.35
Output clamp	±45° (correction authority, not full steering range)
Integral anti-windup clamp	±20°·s

Servo slew limit	12°/tick at the 20Hz control loop
------------------	-----------------------------------

3.10 Speed Control

Vehicle speed is adjusted according to driving conditions.

Examples include:

- Straight sections - Higher speed.
- Sharp turns - Reduced speed to improve stability.
- Obstacle avoidance - Reduced speed to improve positioning accuracy.
- Final approach - Low speed for accurate stopping.

Reducing speed during difficult manoeuvres significantly improves repeatability and reduces the likelihood of leaving the track.

3.11 Obstacle Detection

Obstacle detection is performed using the Raspberry Pi Camera Module.

The perception pipeline performs the following operations:

- Capture image
- Apply colour space conversion
- Threshold red and green colours
- Detect contours
- Identify valid pillars
- Estimate pillar position

Detected obstacles are classified according to colour before being passed to the planning module.

Using colour segmentation provides a computationally efficient solution that is well suited to the fixed colours defined by the WRO rules.

3.12 Obstacle Strategy

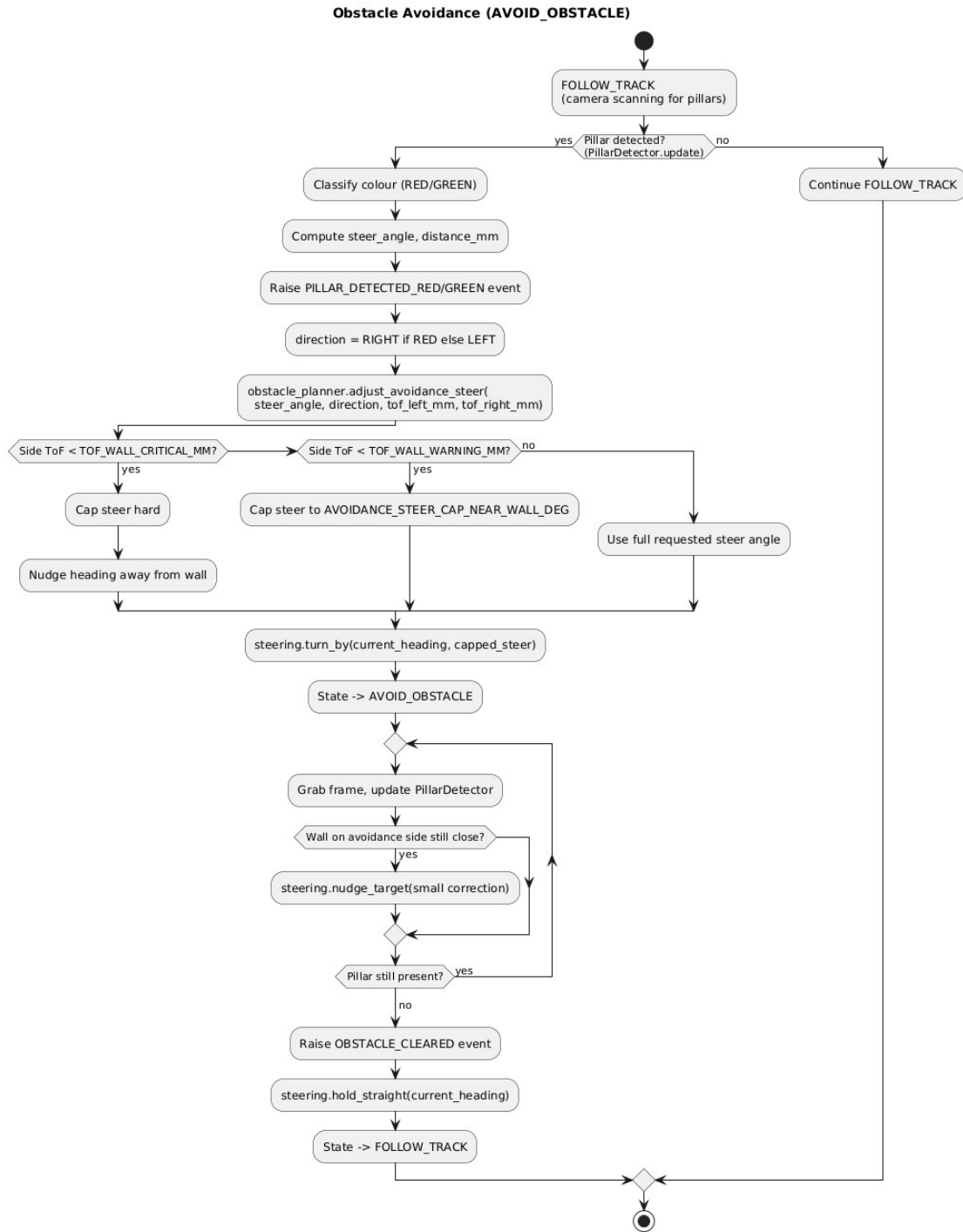
The obstacle strategy follows the WRO Future Engineers competition rules.

When a pillar is detected:

- Red pillar - The robot plans a trajectory that passes on the right side of the pillar.
- Green pillar - The robot plans a trajectory that passes on the left side of the pillar.

Once the obstacle has been safely passed, the robot transitions back to normal lane-following behaviour.

This strategy is implemented using deterministic rules rather than machine learning, making the robot's behaviour easier to verify and reproduce.



3.13 Communication Protocol

The Raspberry Pi communicates with the ESP32 using a serial communication protocol.

The Raspberry Pi sends high-level commands describing the desired vehicle motion.

The ESP32 parses these commands and generates the required PWM signals for the steering servo and drive motor.

The ESP32 validates incoming packets before execution.

Invalid or incomplete packets are ignored to prevent unintended robot movement.

Pi → ESP

Packet	Format
Motion command	CMD,<speed>,<steer>,<action>,<mode>\n
Emergency stop	EMG,<mode>\n
Liveness check	PING\n
Acknowledge	ACK,<tag>\n

ESP→Pi

Packet	Format
Status/log line	STATUS,<message>\n
Telemetry	TEL,<ax>,<ay>,<az>,<gx>,<gy>,<gz>,<tof1_mm>,<tof2_mm>\n
Acknowledge	ACK,<tag>\n
Liveness	PING\n
Hardware event	EVT,<name>\n

All packets are newline-terminated, self-identifying by a leading comma-separated tag (no fixed-width binary framing), and parsing never raises - malformed input degrades to {"type": "unknown"} rather than crashing the read loop.

3.14 Safety Mechanisms

Several software mechanisms improve operational safety.

Serial Watchdog

If communication from the Raspberry Pi stops unexpectedly, the ESP32 automatically stops the motor after a predefined timeout.

This prevents uncontrolled motion in the event of processor failure.

Error State

Critical failures such as invalid sensor data or communication loss cause the robot to enter the ERROR state.

In this state:

- Motors stop.
- Steering commands cease.
- The robot waits for manual intervention.

Defensive Programming

Software modules perform input validation before processing data.

Unexpected values are handled safely rather than causing software crashes.

3.15 Software Testing

Each software module is designed to be tested independently before full system integration.

Metric	Result
--------	--------

Unit test suite (python main.py --mode test)	69/69 passing (grew from 37 baseline as new modules were added)
Simulated multi-tick soak run	300 ticks, zero exceptions, valid JSON status output every tick
Simulated full 3-lap integration test (mocked corner sequence)	Correctly reached PARK via 12 corner events → 3 lap increments → THREE_LAPS_COMPLETE + FINISH_ZONE_DETECTED
Simulated parking-zone integration test (real ParkingZoneTracker, synthetic magenta frame)	Correctly reached PARK after PARKING_CONFIRM_FRAMES (5) consecutive sightings
Synthetic-frame lane detection (symmetric walls)	offset_px $\approx$ 0, confidence 1.0
Synthetic-frame lane detection (asymmetric walls)	Offset sign correctly follows which wall is closer
TOF-aware avoidance steer capping (unit test)	55° requested → 17° actual when side ToF < TOF_WALL_CRITICAL_MM, direction preserved

3.16 Software Design Decisions

Several architectural decisions significantly influenced the final software design.

Deterministic Algorithms

Rather than using machine learning, Hermes uses deterministic computer vision and rule-based decision making.

Advantages include:

- Easier debugging
- Greater reproducibility
- Lower computational requirements
- More predictable behaviour

Distributed Processing

Separating vision from motor control prevents image processing delays from affecting steering performance.

This improves responsiveness and simplifies software development.

Modular Codebase

Each subsystem is implemented as an independent module.

Advantages include:

- Easier maintenance
- Improved readability
- Independent testing
- Simplified future development

3.17 Summary

The Hermes software architecture combines modular design, deterministic algorithms, and distributed processing to provide reliable autonomous operation. A finite state machine coordinates all robot behaviour, while computer vision provides environmental awareness and a PID controller ensures smooth steering. Communication between the Raspberry Pi and ESP32 separates computationally intensive perception from time-critical motor control, improving both reliability and maintainability. The overall architecture was intentionally designed to prioritize robustness, reproducibility, and explainability in accordance with the WRO 2026 Future Engineers documentation rubric.

4. Systems Thinking & Engineering Decisions

4.1 Design Philosophy

Hermes was designed as an integrated engineering system rather than as a collection of independent hardware and software components. Throughout development, every design decision considered how mechanical, electrical, and software subsystems interact to achieve reliable autonomous behaviour.

The team's primary engineering objective was to maximize consistency and repeatability rather than raw speed. The WRO Future Engineers competition rewards robots that complete the challenge reliably and legally, making system robustness more valuable than aggressive performance.

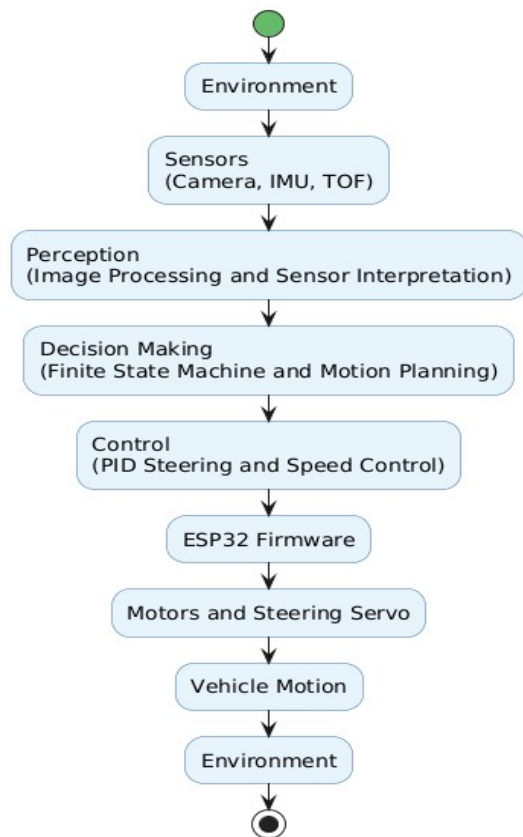
Consequently, the design philosophy adopted throughout the project can be summarized as:

Prioritize reliability, simplicity, and explainability over unnecessary complexity.

This principle influenced every subsystem of the robot.

4.2 Overall System Interaction

Hermes follows a layered system architecture where each subsystem performs a specific role before passing information to the next layer.



This closed feedback loop repeats continuously throughout autonomous operation.

Each layer receives only the information required to perform its task, reducing coupling between modules and improving maintainability.

4.3 Subsystem Responsibilities

To improve modularity, each subsystem has clearly defined responsibilities.

Mechanical System

Responsible for:

- Vehicle stability
- Steering geometry
- Component mounting
- Structural support

The mechanical subsystem provides a stable platform upon which all other systems depend.

Electrical System

Responsible for:

- Power distribution
- Voltage regulation
- Sensor connectivity
- Processor communication

Stable electrical performance is essential for reliable software execution and sensor operation.

Perception System

Responsible for transforming raw sensor data into useful environmental information.

Outputs include:

- Track position
- Lane offset
- Pillar colour
- Finish zone detection

The perception system intentionally provides symbolic information rather than directly controlling actuators.

Decision System

The decision layer determines the robot's behaviour using the finite state machine.

Responsibilities include:

- Selecting operating modes
- Planning obstacle avoidance
- Initiating parking
- Determining lap completion

This layer acts as the "brain" of the robot.

Control System

Responsible for converting desired behaviour into physical motion.

Functions include:

- Steering correction
- Speed regulation
- Motion smoothing

The control system ensures that the robot executes decisions predictably.

Firmware

The ESP32 firmware provides deterministic real-time execution.

Responsibilities include:

- PWM generation
- Servo control
- Motor control
- Safety timeout handling

Keeping these tasks separate from the Raspberry Pi improves overall system reliability.

4.4 Engineering Constraints

Several constraints influenced the design process.

Competition Rules

The robot was required to comply with the WRO Future Engineers regulations, including:

- Four wheels
- One powered axle
- One steering actuator
- Fully autonomous operation
- No wireless communication
- Maximum size restrictions
- Maximum weight restrictions

These constraints limited the range of possible hardware configurations and guided many design decisions.

Computational Resources

Computer vision requires significantly more processing power than real-time motor control.

Rather than forcing a single processor to perform both tasks simultaneously, processing responsibilities were divided between the Raspberry Pi and ESP32.

This reduced processor load and simplified software development.

Reliability

Competition runs provide limited opportunities for recovery from failures.

The robot was therefore designed to minimize:

- Software crashes
- Communication failures
- Mechanical instability
- Electrical resets

Reliability consistently took precedence over performance improvements that introduced additional risk.

4.5 Design Trade-offs

Developing Hermes required balancing competing engineering priorities.

Simplicity vs Complexity

Alternative considered:

Machine learning or deep learning for perception.

Reasons not selected:

- Large training datasets required.
- Increased computational requirements.
- Reduced explainability.
- Greater sensitivity to changing lighting conditions.
- Longer development time.

Selected solution:

Classical computer vision using deterministic image processing.

Advantages:

- Easier calibration.
- Lower computational cost.
- Predictable behaviour.
- Simpler debugging.

Single Processor vs Distributed Processing

Alternative:

Use only a Raspberry Pi for all tasks.

Potential disadvantages:

- Higher processor utilization.
- Less deterministic motor control.
- Increased software complexity.

Selected solution:

Hybrid Raspberry Pi and ESP32 architecture.

Advantages:

- Improved real-time performance.
- Clear separation of responsibilities.
- Easier debugging.
- Greater fault isolation.

Speed vs Reliability

Higher speeds reduce competition completion time but significantly increase:

- Steering oscillation.
- Cornering errors.
- Sensor noise.
- Risk of leaving the track.

The team therefore prioritized reliable completion of the challenge over minimizing completion time.

This decision influenced:

- Speed controller tuning.
- Obstacle handling.
- Final approach behaviour.
- Parking strategy.

4.6 Risk Identification

Several potential risks were identified during development.

Risk	Potential Effect	Mitigation
------	------------------	------------

Poor lighting	Incorrect colour detection	Camera calibration and adjustable thresholds
Serial communication failure	Loss of vehicle control	ESP32 watchdog timeout
Sensor noise	Steering instability	Filtering and validation
Voltage drop	Processor resets	Regulated power distribution
Loose wiring	Intermittent failures	Secure cable routing
Incorrect calibration	Poor autonomous performance	Calibration before testing

Risk identification formed an important part of the engineering design process

4.7 Failure Handling

Rather than assuming perfect operating conditions, Hermes was designed to detect and respond safely to failures.

Examples include:

Communication Loss

If communication between the Raspberry Pi and ESP32 stops unexpectedly:

- Motor commands expire.
- The ESP32 stops the drive motor.
- The robot remains stationary.

Invalid Sensor Data

Unexpected or invalid sensor readings are ignored where possible.

Critical failures cause the state machine to enter the ERROR state.

Software Exceptions

The software architecture isolates major functional modules, reducing the likelihood that failures in one subsystem will propagate throughout the robot.

- Every hardware-adjacent constructor (Camera, KeyboardOverrideListener.start(), SerialLink.connect()) wraps its real-backend attempt in try/except Exception, logs a warning, and degrades to a synthetic/emulated fallback rather than propagating.
- packet_parser.parse_incoming() is documented and implemented to never raise - every malformed input path returns a typed dict ({ "type": "unknown", ... }) instead.
- SerialLink.send()/read_line()/send_emergency() each wrap the actual pyserial I/O call individually, so a write failure doesn't prevent a subsequent read attempt or vice versa.
- run_forever()'s top-level loop catches KeyboardInterrupt separately from generic Exception (clean shutdown log vs. error-with-traceback log), both funnelling into the same finally: self.shutdown().
- shutdown() itself wraps its emergency-stop send in try/except - a failure to notify the ESP32 during shutdown is logged, not allowed to prevent the rest of cleanup (listener stop, camera close) from running.
- Camera.get_frame()/get_jpeg() wrap the actual hardware capture call, logging and returning None/a synthetic frame on failure rather than raising into the perception pipeline.

4.8 Iterative Development Process

Hermes was developed using an iterative engineering approach.

Rather than attempting to complete every subsystem simultaneously, development proceeded through repeated cycles of:

Planning → Implementation → Testing → Evaluation → Refinement

Each iteration focused on improving one subsystem while preserving previously verified functionality.

This incremental process reduced integration problems and simplified debugging.

4.9 Testing Strategy

Testing occurred at multiple levels.

Unit Testing

Individual modules were tested independently.

Examples include:

- Communication
- State machine
- PID controller
- Vision algorithms

Integration Testing

Subsystems were progressively combined to verify interaction between:

- Perception
- Planning
- Control
- Firmware

System Testing

Complete autonomous runs were performed to evaluate:

- Track following
- Obstacle handling
- State transitions
- Overall reliability

Testing results were used to guide subsequent design improvements.

4.10 Engineering Decisions Supported by Testing

Throughout development, design decisions were validated through repeated experimentation rather than assumption.

Examples include:

- Selecting PID parameters based on observed steering behaviour.
- Adjusting sensor placement after testing.
- Refining colour thresholds using camera calibration.

- Modifying steering behaviour to reduce oscillation.

Whenever possible, engineering decisions were based on observed system performance rather than theoretical expectations alone. Some design iterations we had:

1. **TOF driver + lane detection + obstacle avoidance (first pass)** - added the ESP32-side dual-VL53L0X driver, Pi-side camera wall detection, and a ToF-aware cap on the pre-existing pillar-avoidance steering angle.
2. **Correctness pass on the above** - discovered and fixed a sign/state-tracking issue where testing initially showed TOF capping wasn't triggering; traced to test setup overwriting context before the tick that should've used it, not a code bug - but surfaced the need for more careful integration testing going forward.
3. **Lap/corner/finish/parking perception** - added CornerTracker and ParkingZoneTracker, discovered along the way that the pre-existing THREE_LAPS_COMPLETE transition was structurally unreachable (event-queue snapshot semantics meant it was always evaluated one state too late) and fixed the transition table.
4. **Camera synthetic-frame fix** - discovered, while building the lap-counting integration test, that Camera.get_frame() returned None in all simulated runs (only the dashboard JPEG stream had a synthetic fallback), meaning every prior "simulated soak test" claim in this conversation had not actually exercised any vision code path. Refactored the drawing logic into a shared helper so both the JPEG and raw-array paths draw the same scene.

4.11 Future Improvements

Several enhancements have been identified for future development.

Potential improvements include:

- Improved sensor fusion using IMU and Time-of-Flight measurements.
- Adaptive speed control based on track geometry.
- More sophisticated obstacle trajectory planning.
- Enhanced parking accuracy.
- Improved diagnostic and logging capabilities.

These improvements can be incorporated without fundamentally changing the overall system architecture.

4.12 Summary

Hermes was engineered as a cohesive autonomous system in which mechanical, electrical, and software subsystems work together to achieve reliable performance. Every major design decision considered the interactions between subsystems, the constraints imposed by the competition, and the importance of repeatable behaviour. By favouring deterministic algorithms, modular architecture, distributed processing, and iterative testing, the team developed a robot that is robust, explainable, and maintainable. This systems-oriented approach provides a strong foundation for continued refinement while remaining aligned with the objectives of the WRO 2026 Future Engineers challenge.

5. Reproducibility & GitHub Quality

5.1 Project Overview

Hermes was developed with reproducibility as a core objective. The project repository has been structured so that another team can understand the robot's design, reproduce its software architecture, and rebuild the system using the provided documentation, source code, and supporting assets.

The repository contains both the high-level Raspberry Pi software and the low-level ESP32 firmware, allowing the complete autonomous system to be recreated from source.

Development was managed using Git and GitHub, enabling version control, collaboration, and traceability of design decisions throughout the project.

5.2 Repository Structure

The repository is organized into logical modules that separate responsibilities and improve maintainability.

Top-level structure includes:

Firmware/

ESP32 firmware responsible for:

- Motor control
- Servo control
- Serial communication
- Safety watchdog
- Hardware interfaces

```
src/
├── communication/
│   └── Configuration and packet handling.
├── config/
│   └── Robot configuration and system parameters.
├── control/
│   └── Steering control, speed control, and drive command generation.
├── hardware/
│   └── Hardware interface modules.
├── perception/
│   └── Computer vision and environmental sensing.
├── planning/
│   └── Navigation and obstacle planning.
├── simulation/
│   └── Simulation utilities.
├── state_machine/
│   └── Finite State Machine implementation.
├── testing/
│   └── Software testing utilities.
├── utils/
│   └── Shared helper functions.
└── docs/
    └── Project documentation.
```

```
└─ assets/
   └─ Supporting media and design resources.
```

5.3 Software Organization

The software follows a modular architecture in which each module performs one clearly defined responsibility.

This approach provides several advantages:

- Easier debugging.
- Improved readability.
- Simplified maintenance.
- Independent module testing.
- Reduced coupling between components.

For example:

- The perception module processes images but never controls motors.
- The planning module determines desired behaviour but never accesses hardware directly.
- The control module converts desired behaviour into actuator commands without performing computer vision.
- This separation makes the software easier for another team to understand and extend.

5.4 Version Control

Git was used throughout development to record software changes and coordinate collaboration between team members.

Version control provides:

- Complete history of software development.
- Ability to recover previous versions.
- Safe experimentation using branches.
- Easier collaboration through pull requests.

The team adopted the following workflow:

- Create feature branches for new work.
- Develop and test features independently.
- Merge completed work into the main branch using pull requests.
- Synchronize repositories using Git Fetch and Git Pull before beginning new work.

This workflow minimizes merge conflicts and improves software quality.

5.5 Commit History

Meaningful commit messages were used to document project progress. And changes were made via branch and Pull Requests, never directly to the main branch.

Rather than committing unrelated changes together, commits represent individual engineering tasks or feature additions.

Examples include:

- feat: perception
- feat: lane following

- chore: clean up comments
- fix: duplicate method in state machine

A well-maintained commit history allows the development process to be reconstructed and demonstrates continuous progress throughout the project.

5.6 Documentation

The project documentation has been prepared to allow another team to understand both the implementation and the engineering reasoning behind the robot.

Documentation includes:

- Mechanical design.
- Power architecture.
- Sensor placement.
- Software architecture.
- System interactions.
- Engineering decisions.
- Testing methodology.
- GitHub repository information.

Where appropriate, diagrams are included to improve clarity and reproducibility.

Documentation emphasizes not only *what* was built, but also *why* each design decision was made.

5.7 Supporting Files

The repository includes all resources necessary to understand the robot's design.

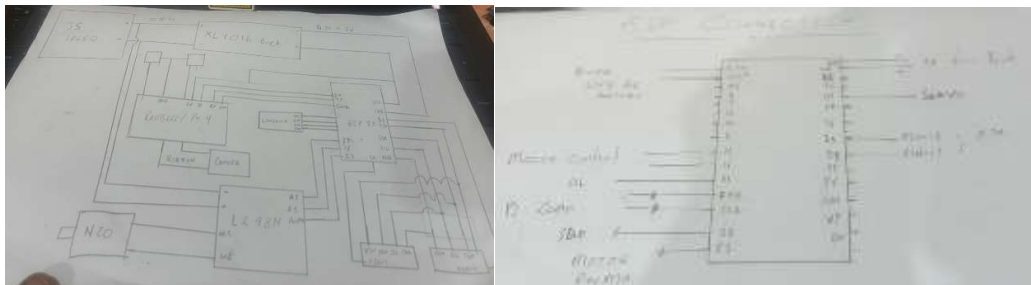
Supporting materials include:

- Source code.
- Firmware.
- Configuration files.
- Documentation.
- CAD files.
- Wiring diagrams.
- Testing utilities.

These resources collectively provide sufficient information for another team to reproduce the system.

Even CAD models are present in the /models directory.

Wiring Diagrams (System and ESP)



5.8 Build and Setup Process

A new user can reproduce the software by following the documented setup process.

Typical setup includes:

1. Clone the repository.
2. Install required software dependencies.
3. Configure the Raspberry Pi.
4. Upload firmware to the ESP32.
5. Connect hardware according to the wiring diagram.
6. Calibrate sensors.
7. Verify serial communication.
8. Execute autonomous software.

Providing a structured setup process reduces ambiguity and improves reproducibility.

5.9 Hardware Reproducibility

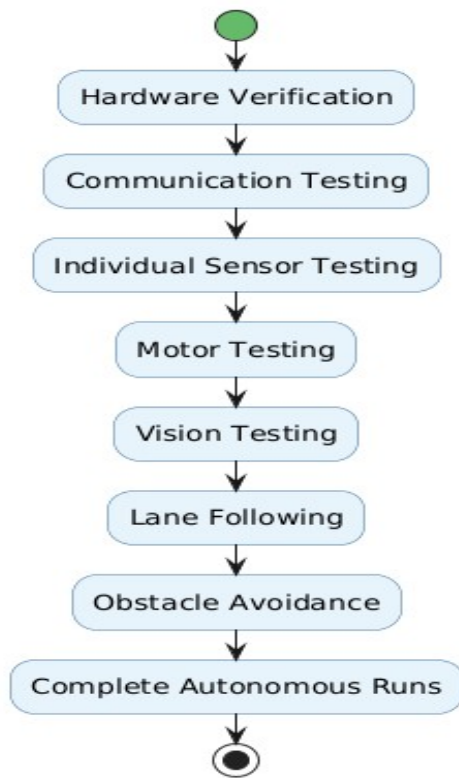
The hardware design has been documented to enable reconstruction by another team.

This includes:

- Bill of Materials.
- Component descriptions.
- Sensor placement.
- Power architecture.
- Mechanical layout.

Where custom parts are used, CAD models are included in the repository.

5.10 Testing Workflow



5.11 Reproducing the Robot

A team attempting to recreate Hermes should follow the following sequence:

1. Assemble the chassis.
2. Install all electronic components.
3. Complete power wiring.
4. Upload ESP32 firmware.
5. Configure the Raspberry Pi software.
6. Install software dependencies.
7. Connect the Raspberry Pi and ESP32 through serial communication.
8. Calibrate camera and sensors.
9. Verify steering and motor operation.
10. Test lane following.
11. Test obstacle avoidance.
12. Perform complete autonomous runs.

Following this sequence minimizes integration problems and allows faults to be isolated more effectively.

5.12 Future Repository Improvements

Several improvements have been identified that would further enhance reproducibility.

These include:

- Automated installation scripts.
- Dependency management documentation.

- Release version tagging.
- Continuous integration testing.
- Expanded troubleshooting documentation.
- Hardware calibration checklists.
- Additional configuration examples.

These improvements would simplify onboarding for future team members and external users.

5.13 Summary

Hermes was developed with reproducibility as a fundamental design objective. A modular repository structure, meaningful version control practices, comprehensive documentation, and clearly separated software components allow another team to understand, rebuild, and extend the robot with minimal ambiguity. By documenting both the implementation and the engineering decisions behind it, the project satisfies the goals of transparency, maintainability, and reproducibility expected by the WRO 2026 Future Engineers documentation rubric.

Appendix A - Testing and Design Iterations

A.1 Development Methodology

The development of Hermes followed an iterative engineering process in which each subsystem was designed, tested, evaluated, and refined before being integrated into the complete robot. This approach allowed faults to be isolated more effectively and ensured that improvements in one subsystem did not unintentionally reduce the performance of another.

Rather than attempting to complete the entire robot in a single development cycle, the team adopted a continuous improvement workflow consisting of:

Planning → Implementation → Testing → Evaluation → Refinement

Each iteration focused on improving a specific subsystem while maintaining the overall stability of the robot.

A.2 Mechanical Iterations

Several revisions were made to the mechanical design throughout development.

Final Implementation

Objective:

Produce the competition-ready configuration.

Final improvements included:

- Improved structural rigidity.
- Improved component accessibility.
- Cleaner cable routing.
- Stable sensor mounting.

A.3 Software Development Timeline

Software development progressed in stages rather than being developed simultaneously.

1. Architecture phase (early) - Pi/ESP32 ownership split established, I2C sensor addresses documented, ESP32 pin map drafted, Python module structure (serial comms, packet parsing, state machine, drive commands, logging) laid out.
2. Core comms + state machine build - serial protocol, packet parser, event queue, state transitions, RobotContext; git branch corruption (main vs feat/DriveCommand) resolved along the way.
3. State machine audit - checked against competition spec for correctness.
4. Firmware serial layer refactor - Pi-side serial layer rewritten, matching typed ESP32 firmware endpoints; a telemetry function naming conflict resolved.
5. Motor control implemented, servo control verified, PWM output tested.
6. Camera integrated, Vision pipeline implemented.
7. Firmware driver, camera-based lane detection, TOF-aware obstacle-avoidance capping.
8. Lap/corner counting, finish-zone and parking-zone detection, state-machine transition bugfix.
9. Camera simulated-frame fallback fix, enabling vision code to actually run under simulated=True.
10. Full autonomous state machine completed.

A.4 Communication Testing

Reliable communication between the Raspberry Pi and ESP32 was critical to the robot's operation.

Testing focused on:

- Packet transmission.
- Packet parsing.
- Response timing.
- Invalid packet handling.
- Communication recovery.

During development, several communication issues were encountered and resolved.

Examples included:

- Incorrect UART configuration.
- Packet synchronization issues.
- Timeout handling.
- Serial buffer management.

Solutions implemented included:

- Standardized packet formatting.
- Packet validation before execution.
- Communication timeout detection.
- Automatic motor shutdown if communication is lost.

A.5 PID Steering Tuning

The steering controller was tuned experimentally.

Each adjustment involved:

1. Selecting new PID gains.
2. Driving multiple laps.
3. Observing oscillation.
4. Recording behaviour.
5. Refining controller parameters.

Performance was evaluated according to:

- Steering smoothness.
- Overshoot.
- Cornering accuracy.
- Lane-centreing performance.

Final PID gains

Parameter	Value
STEER_PID_KP	1.6
STEER_PID_KI	0.02
STEER_PID_KD	0.35
Output clamp	$\pm 45^\circ$ (correction authority, not full steering range)
Integral anti-windup clamp	$\pm 20^\circ \cdot s$
Servo slew limit	12°/tick at the 20Hz control loop

A.6 Vision Testing

Computer vision was evaluated under multiple lighting conditions.

Testing considered:

- Bright lighting.
- Indoor lighting.
- Shadows.
- Variable reflections.

Calibration focused on:

- Colour thresholds.
- Exposure.
- White balance.
- Region-of-interest placement.

These adjustments improved the consistency of obstacle detection and lane estimation.

A.7 System Testing

Full-system testing evaluated the interaction between all subsystems.

Testing scenarios included:

- Straight-line driving.
- Cornering.
- Obstacle avoidance.
- Lap counting.
- Finish detection.

Each test was repeated multiple times to evaluate repeatability.

Performance metrics were recorded.

A.8 Lessons Learned

Several important engineering lessons were identified during development.

Examples include:

- Modular software significantly simplified debugging.
- Separating perception and control reduced software complexity.
- Reliable communication is more important than communication speed.
- Mechanical alignment greatly influences software performance.
- Small hardware adjustments can significantly improve autonomous behaviour.

Appendix B - Mechanical Drawings, Wiring and CAD References

B.1 Mechanical Drawings

The repository contains the CAD models used during development.

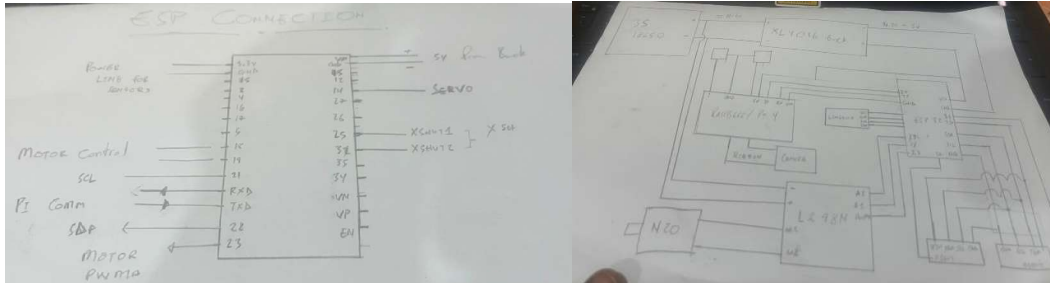
These models describe:

- Chassis geometry.
- Mounting brackets.
- Sensor mounts.

- Structural components.

For brevity, the complete CAD models are provided in the project repository rather than embedded within this document.

B.2 Wiring Diagram



B.3 Sensor Placement

The sensor arrangement was selected to maximize environmental awareness while minimizing interference.

Camera: Front-mounted with forward-facing field of view.

IMU: Mounted near the centre of mass.

Time-of-Flight Sensors: Mounted on the sides of the vehicle for lateral distance measurement.

B.4 Mechanical Assembly

The robot was assembled using a modular approach.

Major assembly stages included:

1. Chassis assembly.
2. Steering installation.
3. Drive motor installation.
4. Battery installation.
5. Sensor mounting.
6. Electronics installation.
7. Wiring.
8. Software upload.
9. Calibration.

Appendix C - Bill of Materials

Materials	Description	quantity	cost	URL
VL53L1X (TOF)	Measuring distance	4	GHC 67.03	1/3/5pcs VL53L0X Time-of-Flight (ToF) Laser Ranging Sensor Breakout
Raspberry pi 4	Process data from Pi-Cam	1	Covered	Covered
Pi-Cam 3	For computer vision	1	GHC 98.10	Raspberry Pi Camera Module
ESP-WROOM-32	Sensor integration and motor control	1	GHC 64.00	N/A

DC Motor	Bot Movement	1	GHC 85.00	DC12V N20 600rpm Micro Metal Gear Motor
DRV8833 Driver	Controlling motor direction and speed	1	GHC 85.93	Set of 4 DRV8833 Motor Driver Modules,
18650 2000mAh 3.7V Rechargeable Lithium Battery	Power Supply	2	GHC 50	3.7V 18650 Lithium ion Battery Single 2550mah
Battery holder	Contain the batteries	1	GHC 28.00	2 x 18650 Battery Holder with Cover & ON/OFF Switch
Power switch	Turn on/off the bot.	1	GHC 2.00	2Pin SPST Mini Boat Rocker Switch 3A 250V Red
Charging Module	Charge the batteries	1	GHC 36.27	Dual-slot 18650 Lithium Battery Charger, 4.2V Smart Fast Charging, Black Universal Charging Dock
Buck converter 7.4 V $\rightarrow$ 5 V (3 A+)	Regulate Voltage	1	GHC 18.00	LM2596 LM2596S DC-DC Step- Down Buck Converter Module with SMD LED
MPU6050	Gyroscope/Motion Sensing	1	GHC 42.00	MPU-6050 3 Axis Acceleration + Gyro Breakout
MG90S Servo	Position-controlled movement	1	GHC 38.00	MG90S – Metal Gear Micro Servo Motor
LSM6DSOX IMU	Movement and orientation tracking	1	Covered	Covered

C.1 Estimated Cost

We invested approximately GHC 600.00 and a lot of time into this project.

C.2 Component Selection Rationale

Each major component was selected according to engineering requirements rather than availability alone.

- **Raspberry Pi 4:** Provides sufficient processing power for computer vision and high-level planning.
- **ESP-WROOM-32:** Offers reliable real-time control, PWM generation, and serial communication.
- **Camera Module:** Enables lane and obstacle detection using computer vision.
- **MG90S Servo:** Provides accurate and repeatable steering control.
- **Time-of-Flight Sensors:** Supply direct distance measurements that complement vision.
- **IMU:** Provides orientation and motion data for improved state estimation.
- **Lithium-ion Battery Pack:** Offers a high energy density suitable for repeated autonomous runs.